

CAPSTONE PROJECT PROPOSAL (Draft v1.0)

Project Title:

SentinelAI: An AI-based Autonomous Security Agent for Threat Detection and Response in SOC and Data Center Environments

Student: Nguyễn Đức Bình

Group: Group 5

Course: Capstone Project

1. Introduction and Background

1.1 Problem Description

Modern Security Operation Centers (SOC) and data center environments generate an enormous amount of security alerts from systems such as SIEM, IDS/IPS, EDR, and cloud monitoring tools. Security analysts must manually investigate these alerts to determine whether they represent real threats or false positives. This process is time-consuming and often inefficient.

According to recent studies, a large proportion of security alerts are false positives, leading to alert fatigue among analysts and delayed incident responses. This situation becomes more critical in environments such as data centers where attacks such as lateral movement, privilege escalation, and data exfiltration can occur quickly.

Existing security automation solutions mainly rely on rule-based detection or static playbooks. These solutions often lack contextual understanding and cannot effectively correlate multiple security events into meaningful attack scenarios.

Therefore, there is a need for a more intelligent approach that can assist security analysts in identifying real threats, correlating alerts, and recommending appropriate responses.

Developing an AI-based security agent that can automatically observe system events, analyze suspicious behavior, and assist SOC analysts in responding to threats may significantly improve incident detection and response capabilities.

1.2 Research Objectives

Main Objective

The main objective of this project is to design and develop an AI-based autonomous security agent that assists SOC analysts in detecting and responding to cybersecurity threats in data center environments.

Specific Objectives

- Analyze current approaches used for threat detection and SOC automation.
- Design an AI-based security agent capable of correlating security alerts into meaningful incidents.
- Develop a prototype system that assists analysts in analyzing threats and recommending responses.
- Evaluate the effectiveness of the proposed system in reducing alert noise and improving incident response efficiency.

Research Questions

1. What are the existing methods used in SOC automation and threat detection systems, and what are their advantages and limitations?
2. What AI-based approach can be proposed to improve threat detection and incident response?
3. How can the effectiveness of the proposed solution be evaluated in terms of detection performance and operational efficiency?

1.3 Scope of the Project

Included in the project

- AI-based analysis of security alerts
- Correlation of security events into incidents
- Threat detection using behavioral analysis
- Decision support for incident response
- Integration with simulated SOC environment logs

Excluded from the project

- Full enterprise-scale SOC deployment
- Offensive penetration testing
- Real-time protection for national-scale infrastructure

The focus of the project is on building a prototype system that demonstrates the feasibility of AI-assisted threat detection and response.

2. Proposed Solution

2.1 Solution Description

This project proposes **SentinelAI**, an AI-based autonomous security agent designed to assist SOC analysts in monitoring and responding to cybersecurity threats.

The system continuously collects security events from multiple sources such as SIEM logs, network events, endpoint alerts, and authentication logs. These events are processed and analyzed using machine learning and rule-based reasoning techniques.

SentinelAI correlates multiple alerts into meaningful incidents, identifies suspicious patterns, and assigns risk scores to detected activities. Based on predefined security policies, the system can recommend appropriate responses or trigger automated actions with human supervision.

The proposed system aims to improve SOC operational efficiency by reducing alert noise, improving threat visibility, and assisting analysts in responding to incidents more effectively.

2.2 Software Architecture (General)

The proposed system follows a modular architecture that integrates with existing SOC components.

Architecture Overview

Security Data Sources

(SIEM, IDS/IPS, EDR, IAM, Cloud Logs)

|

v

Data Ingestion Layer

(Log collection & normalization)

|

v

SentinelAI Agent

| Event Observation Module |

| Alert Correlation Engine |
| AI Reasoning & Analysis |
| Risk Scoring & Decision |
| Response Recommendation |
| Learning & Memory Module |

|

v

SOC Analyst / Response System

(Alerting, Incident Handling)

Component Roles

Data Ingestion Layer

Collects and normalizes security logs from various sources.

Alert Correlation Engine

Combines related alerts into meaningful incidents.

AI Reasoning Engine

Uses machine learning and statistical analysis to identify suspicious behaviors.

Decision Engine

Evaluates risk levels and determines possible response actions.

Response Module

Suggests mitigation actions or escalates incidents to SOC analysts.

Learning Module

Stores historical incidents and feedback to improve system performance.

2.3 Technology and Tools Used

The following technologies are proposed for the development of the system:

- **Python** – core system development
- **Scikit-learn / PyTorch** – machine learning models

- **FastAPI** – backend service development
- **Elastic Stack (ELK)** – log ingestion and analysis
- **PostgreSQL** – data storage
- **Docker** – containerized deployment

These technologies are selected because they are widely used in security analytics and provide strong support for data processing and machine learning applications.

3. Implementation Plan

3.1 Main Development Stages

The project will be developed through the following phases:

Phase 1 – Requirement Analysis

Study SOC workflows, threat detection mechanisms, and system requirements.

Phase 2 – System Design

Design the architecture of SentinelAI and define the system modules.

Phase 3 – Development

Implement the AI agent modules including event processing, correlation, and threat detection.

Phase 4 – Testing

Test the system using simulated security logs and evaluate detection performance.

Phase 5 – Documentation

Prepare the final project report and system documentation.

3.2 Expected Schedule (Preliminary)

Phase	Timeline
Requirement Analysis	Month 1
System Design	Month 1

Development	Month 2-3
-------------	--------------

Testing	Month 4
---------	---------

Documentation	Month 5
---------------	---------

3.3 Feasibility Assessment

Technical Challenges

- Limited access to real SOC data
- Difficulty in accurately identifying complex attack patterns

Mitigation Strategies

- Use publicly available cybersecurity datasets
- Simulate SOC environments for testing

Required Resources

- Personal development workstation
- Public cybersecurity datasets
- Open-source security monitoring tools

These resources are accessible and suitable for the development of a prototype system.

4. Expected Results

Upon completion, the project is expected to deliver:

- A functional prototype of an AI-based security agent
- A system capable of correlating security alerts and identifying potential threats
- Improved efficiency in analyzing security events compared to manual analysis
- Documentation and system architecture for further development

5. Evaluation Plan

The effectiveness of the proposed system will be evaluated based on several criteria:

Detection Accuracy

Ability to correctly identify suspicious events.

Alert Reduction Rate

Reduction of unnecessary alerts through correlation and filtering.

Incident Response Efficiency

Improvement in response time for detected incidents.

System Usability

Ease of use for SOC analysts.

Evaluation will be conducted using simulated security datasets and experimental testing.

6. References

[1] É. Lefèvre, "AI-Based Modeling in Software Engineering: Techniques, Applications, and Future Directions," Oct. 2022.

[2] M. Osaka and D. Karan, "AI-Powered Testing Framework for Quality Assurance in Software Systems," Oct. 2024.

[3] D. Karan and A. Ethan, "Leveraging AI for Predictive Risk Assessment in Software Systems," Dec. 2024.